End of Day Report

Name	Atibali Saiyed
Date	04 November 2026
Training Day	Day 2 — Part 1: C# & SQL Fundamentals · Part 2: QA Automation Mindset
Repository	https://github.com/Atibali/Gateway-Training/tree/main/Week1/Day2

Part 1 — C# Fundamentals

Exceptions

Exception handling — Learned how C# handles runtime errors through `try/catch/finally`. The `try` block wraps risky code, `catch` handles specific exception types (ordered from most specific to most general), and `finally` executes cleanup code. Learned to throw meaningful exceptions, avoid silently swallowing them, and use `throw;` rather than `throw ex;` to preserve the original stack trace.

Common and custom exceptions — Covered common built-in types including `ArgumentException`, `InvalidOperationException`, `NullReferenceException`, `FormatException`, `IndexOutOfRangeException`, `DivideByZeroException`, and I/O exceptions. Also learned to create custom exceptions by deriving from `Exception` and providing standard constructors for domain-specific errors.

Dates and Times

DateTime and DateTimeOffset — Understood the difference between `DateTime` (with `Kind` values `Utc`, `Local`, or `Unspecified`, but no explicit offset) and `DateTimeOffset` (a point in time with an explicit UTC offset, making the timestamp unambiguous). Learned the practice of storing and transporting timestamps in UTC and converting them to local time only for display.

Conversions and durations — Practised UTC-to-local and local-to-UTC conversions, parsing with time-zone awareness, and using `TimeSpan` for durations. Created durations with `FromMinutes` and `FromHours`, calculated durations by subtracting dates, and used properties such as `TotalHours`.

Part 1 — SQL (sqlpractice Database)

Relational Basics

Tables, keys, and relationships — Reinforced that relational tables contain rows and columns with defined data types such as `INT`, `VARCHAR`, `DATE`, and `DECIMAL`. Primary keys uniquely identify rows and cannot be `NULL`; foreign keys enforce referential integrity across relationships such as `orders.CustomerID → customers`, `orderitems.OrderID → orders`, and `orderitems.ProductID → products`. Observed how foreign-key constraints prevent orphan records and invalid inserts.

Reading Data

SELECT and filtering — Practised `SELECT` with aliases, computed columns, and string/date functions. Applied `WHERE` with `AND`, `OR`, and `NOT` (including operator precedence); `LIKE` patterns (`%`, `_`, and escaping); `IN` (including subqueries); `BETWEEN` (inclusive ranges, with date-range best practices); `DISTINCT`; and `LIMIT … OFFSET` for pagination.

Sorting and conditional values — Used multi-column `ORDER BY`, `CASE` expressions for conditional columns, and `COALESCE` / `IS NULL` for handling missing values.

Aggregates

Grouping and aggregate functions — Used `COUNT(*)`, `COUNT(column)`, `COUNT(DISTINCT …)`, `SUM`, `AVG`, `MIN`, and `MAX` with `GROUP BY`. Learned the key distinction: `WHERE` filters rows before grouping, while `HAVING` filters groups after aggregation.

Advanced aggregation — Practised conditional aggregation with `CASE`, multi-column `GROUP BY`, aggregates over expressions, and window functions such as `SUM() OVER` and `RANK() OVER (PARTITION BY …)`.

Joins

Join types and multi-table joins — Covered `INNER JOIN` (matching rows only), `LEFT JOIN` (all left-side rows plus matches), and `RIGHT JOIN` (the mirror of `LEFT JOIN`). Practised chained three- and four-table joins across `customers → orders → orderitems → products`.

Advanced join patterns — Learned the anti-join pattern (`LEFT JOIN … WHERE child.pk IS NULL`) to find missing relationships, self-joins, joins on derived tables, and the important rule that right-side predicates on a `LEFT JOIN` should generally be placed in `ON`, not `WHERE`, to avoid unintentionally turning it into an inner join. Also covered `UNION` versus `JOIN` and `CROSS JOIN`.

Changing Data

INSERT, UPDATE, and DELETE — Practised `INSERT` with explicit column lists and defaults, `UPDATE` with targeted `WHERE` clauses (including stock decrements and bulk status changes), and `DELETE` for selected rows. Observed foreign-key restrictions when attempting to delete parent rows.

Creating Tables and Constraints

Table design and validation — Created a `reviews` table using `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK (Rating BETWEEN 1 AND 5)`, `DEFAULT CURRENT_TIMESTAMP`, and `FOREIGN KEY … ON DELETE CASCADE`. Verified the constraints by testing both valid and invalid inserts.

Design Basics

Normalisation — Learned the principles of 1NF (atomic values and no repeating groups), 2NF (no partial dependencies on a composite key), and 3NF (no transitive dependencies), and how normalisation reduces duplication and update anomalies.

Indexes and query performance — Understood indexes as B-tree structures that can speed up `WHERE`, `JOIN`, and `ORDER BY` operations at the cost of storage and write performance. Reviewed existing primary-key, unique-email, and foreign-key indexes; practised adding composite indexes, checking query plans with `EXPLAIN`, and applying the leftmost-prefix rule. Learned to avoid wrapping indexed columns in functions when doing so prevents efficient index use.

Part 2 — QA Automation Mindset

What Automation Is For

Purpose of test automation — Learned that automation uses code to verify software behaviour without requiring a person to perform every check. Its benefits include fast regression detection, CI/CD feedback, freeing testers for exploratory work, living documentation, and confidence when refactoring. Understood automation as an investment rather than simply a cost.

What to Automate vs. What Not To

- Automate repeatable, stable, high-regression-risk, data-driven, deterministic, CI-executed, and compliance-related checks.
- Avoid automating one-off tests, exploratory testing, subjective UX/look-and-feel judgments, rapidly changing features, flaky areas, and checks whose maintenance cost exceeds their defect-detection value.

The Test Pyramid

- Unit tests (base, approximately 70%): fast, inexpensive, stable, and focused on one function; they identify logic defects precisely.
- API / Integration tests (middle, approximately 20%): validate service contracts and database boundaries at moderate speed.
- UI / End-to-End tests (top, approximately 10%): cover only critical user journeys because they are slower and more brittle.

Anti-patterns and alternatives — Learned about the Ice Cream Cone anti-pattern (too many UI tests) and why tests should be pushed down the pyramid where practical. Also became aware of alternatives such as the Testing Trophy and Honeycomb, while treating the test pyramid as a useful default cost-and-speed model.

Return on Investment (ROI)

Measuring automation value — Learned the formula ROI = (value gained − cost) / cost. Value comes from repeated execution and earlier defect detection; costs include test development, maintenance, and infrastructure. Break-even runs can be estimated as build cost ÷ manual run cost.

Factors affecting ROI — ROI improves with frequent execution, stable tests, high-risk coverage, data-driven design, and shift-left practices. It decreases with flaky tests, brittle selectors, over-automation, slow suites, and poor test design. Understood shift-left testing (testing early through unit and API checks) as a major ROI lever because defect-fixing costs rise as issues move from development to CI, QA, and production.

Summary

Today I learned C# exception handling using try/catch/finally, custom exceptions, and the fundamentals of date and time management using DateTime, DateTimeOffset, and TimeSpan. I also strengthened my SQL knowledge, covering relational database concepts, queries, filtering, aggregate functions, joins, data manipulation, constraints, normalization, and indexing. Additionally, I learned the QA automation mindset, including what to automate, the Test Pyramid, automation ROI, and shift-left testing practices.